

GeoPulse

End-to-end ML system: 79M-row spatiotemporal forecasting pipeline, four model families, and an agentic map application

Python · Polars · DuckDB · LightGBM · PyTorch · FastAPI · MapLibre · 79,410,195 trips · 11,238 lines of code · 166 tests passing

1. What it does

New York's Citi Bike has ~2,400 docking stations. Bikes pile up where people ride *to* and vanish from where people ride *from* — every weekday morning thousands flow into Midtown, leaving residential docks empty. **GeoPulse predicts that flow before it happens**, then makes the forecast usable: a map app where you ask "will I find a bike near Times Square?" and get a real answer with walking directions and an explanation.

1.14

TEST MAE (BIKES / 15 MIN)

38%

BETTER THAN BASELINE

0.72

HOTSPOT F1

166

TESTS PASSING

Technical stack

Layer	Technologies and why
Data engineering	Polars (streaming CSV→Parquet, 15 GB → 2.5 GB in 2 min), DuckDB (out-of-core SQL, window functions, hive-partitioned writes), Parquet throughout — pandas deliberately avoided at scale
Spatial	Uber H3 (hexagonal indexing, resolutions 8–10), Google S2 (level 13), custom <code>SpatialIndexer</code> protocol so both are interchangeable
Modelling	LightGBM (Poisson objective, Optuna tuning), PyTorch — hand-written Temporal Fusion Transformer and Spatio-Temporal GNN with graph attention via edge-list segment softmax
Serving	FastAPI , Pydantic, in-process model/tensor caching, tool-calling agent over Mistral / Claude APIs with a dependency-free deterministic fallback
Frontend	MapLibre GL JS with vector basemaps, hand-written CSS, no framework — map-first layout with a sliding bottom sheet
Algorithms	A* with an admissible haversine heuristic over an OpenStreetMap walking graph; greedy constrained rebalancing simulation
Engineering	pytest (166 tests incl. 35 leakage tests), YAML-driven configuration, phase gates verified by scripts against artifacts, git history one commit per phase

2. The four research questions

#	Question	Answer
RQ1	How big should a map cell be?	H3 resolution 8 (~0.74 km²). Chosen by <i>skill score</i> , because every ordinary metric is biased by cell size and the three we tried disagreed completely.
RQ2	Hexagons or squares?	Near coin flip . S2-13 scores slightly higher skill, but its cells are 47% larger — which plausibly explains the whole gap. Size matters; shape barely does.
RQ3	Do deep models beat gradient boosting?	No . Tuned LightGBM won at both grids. Two apparent deep-learning wins at H3-8 reversed at S2-13, so they were noise. Caveat in §7.
RQ4	Do better forecasts improve operations?	Yes between forecasts, no in absolute terms . LightGBM beat even a <i>perfect</i> forecast at rebalancing — but rebalancing itself didn't help. §8.

3. Pipeline

90 CSVs (15 GB) → Parquet (2.5 GB, 2 min) Polars streaming, never pandas

- Clean: 9 named rules, 245,128 removed (0.309%), counts reconcile exactly
- Timestamps → UTC (daylight-saving safe), 4 distorted hours flagged
- Index 79M trips → H3 cells in 0.8 s (only 3,230 distinct coordinates exist)
- Panel: 104,071,008 rows = 1,483 cells × 70,176 intervals, exactly dense
- Features: 168 columns across 8 families, 15 leakage tests
- Models: Seasonal Naive · LightGBM · TFT · ST-GNN, × 2 spatial systems
- TEST (Nov-Dec 2024) opened exactly once → App

Each phase had a written Definition of Done, verified **by scripts reading the actual output files** — not by ticking boxes. That caught several bugs that would have silently corrupted everything downstream (\$9).

4. Data and cleaning

Four sources: **Citi Bike trips** (90 monthly CSVs), **weather** (Open-Meteo hourly, 17,544 rows, no gaps), **events** (557,674 NYC permit records) and **traffic** (Socrata API, 23,758,184 readings).

A FALSE START WORTH RECORDING

The traffic CSV downloaded from the NYC Open Data web UI opened cleanly and looked correct. It had been **silently row-capped** and contained only five days from July 2018 — zero overlap with the project window. Caught only because a check compared the file's date range against the project's. Refetched via the Socrata API, paginating by time window rather than row offset, and completeness was *proved* against a server-side `count(1)`.

The cleaning waterfall

Nine named rules, applied in order; each removed row attributed to the **first** rule it violates, so counts sum exactly to the total. A quality report is produced *before* anything is deleted.

Rule	Removed	Rule	Removed
duplicate Ride_id	0	duration_too_long (>24 h)	37,627
null Ride_id / null timestamp	0	missing_coords	206,222
outside_project_window	276	coords_out_of_bbox	178
end_before_start	643	Total removed	245,128 (0.309%)
duration_too_short (<60 s)	182	Kept	79,165,067

Daylight saving — the problem nobody expects

Citi Bike publishes **naive local time**. In November the 1–2 am hour happens twice, written identically both times; bin it in local time and one interval gets double demand. In March that hour doesn't exist. Everything is stored in **UTC** instead, and the damage was *measured* rather than assumed:

UTC hour	Rides	Meaning
2023-11-05 05:00	2,921	Both passes collapsed together
2023-11-05 06:00	0	The unreachable second pass
2024-11-03 05:00 / 06:00	3,534 / 0	Same pattern next year

Four affected hours are flagged `dst_unreliable` and excluded from every model, so nothing learns the artificial collapse.

Sorting 79 million rows

A global `ORDER BY` spilled 8 GB and hadn't finished after an hour. Because the data is already partitioned by month, sorting each month (~3M rows, in memory) and concatenating in month order gives the *identical* ordering — 90 seconds. Verified two independent ways: **0 inversions across 79,165,067 rows**.

5. The panel and the features

The panel is **exactly one row per cell per 15-minute interval**, no gaps. A cell with no trips gets a **0**, not a missing row — because "four rows back" only means "one hour ago" if every interval is guaranteed present.

Property	Value
Rows	104,071,008 = 1,483 cells × 70,176 intervals — exactly dense
Reconciliation	Panel pickups 79,150,890 — matches trips in active cells exactly
Active cells	1,483 of 1,570 touched, carrying 99.98% of all endpoints
Targets	15/30/45/60 min ahead × pickups/dropoffs, re-derived by independent join
End of series	NULL, never 0 — a zero teaches a phantom demand collapse

Preventing leakage

Forecast time is pinned to the END of an interval. That makes a one-step forecast genuinely 15 minutes ahead, and makes the just-completed interval a legal input — the single most informative feature, which an earlier phase had omitted.

PROOF, NOT INSPECTION

15 tests **re-derive every lag feature by an independent timestamp self-join** and assert agreement; 20 more cover the advanced families. Reading code catches the bugs you thought of; re-deriving the answer a different way catches the ones you didn't.

Which features earned their place — the ablation

Family added	Features	Pickup MAE	Change	Verdict
A — recent demand	21	1.6944	—	baseline
B — calendar/seasonal	76	1.5791	+6.60%	Helps a lot
C — rolling trends	125	1.5695	+0.64%	Helps
D — weather	142	1.5703	−0.05%	No benefit
E — events	148	1.5702	−0.01%	No effect
F — traffic	155	1.5702	−0.05%	No effect
G — neighbours	164	1.5675	+0.30%	Slight
H — station network	168	1.5684	−0.07%	No benefit

Final feature set: A + B + C, 125 features. Weather, events and traffic contribute essentially nothing — nearly all signal beyond recent demand is calendar and seasonality, which is intuitive for commuter cycling and cheap.

TRAFFIC GOT A FAIR SECOND CHANCE

The first ablation ran at H3-9, where traffic sensors reach only **5.1% of cells** — that tests "traffic in 5% of Manhattan", not "traffic". At H3-8 coverage is **20.2%, four times better**, so the whole ablation was re-run. **Quadrupling coverage changed nothing** (still −0.05%). That turns a weak result into a defensible negative finding.

Gain-based importance from the trained model agrees independently: the top five features are `pickups_ewm_4`, `pickups_roll_mean_4`, `pickups_lag_0`, `pickups_slot_expanding_mean`, `pickups_roll_sum_4` — all from families A, B and C.

6. Choosing the cell size — and why the obvious metric lies

Resolution	Cells	Area km ²	% zero	MAE	WAPE	Hotspot F1	Skill
H3-8	321	0.741	47.5%	1.654	0.308	0.748	0.2563
H3-9	1,483	0.106	71.4%	0.705	0.605	0.590	0.1920
H3-10	2,162	0.015	75.6%	0.580	0.731	0.481	0.1791

MAE says resolution 10. WAPE and Hotspot F1 say resolution 8. That total disagreement is the clue that none of them measures skill:

- **MAE falls automatically as cells shrink** — smaller cells hold smaller counts. In the limit, a cell that always contains zero has perfect MAE and zero use.
- **WAPE falls automatically as cells grow** — aggregation smooths relative error. In the limit, one cell covering all of NYC scores beautifully and says nothing.
- **Hotspot F1 depends on cell count** — top 10% of 321 is a different task from top 10% of 2,162.

The fix is a **skill score**: $1 - \text{model_MAE} / \text{naive_MAE}$, both computed on the same grid, so the scale effect cancels.

Resolution 8 wins, and also wins on sparsity (47.5% empty vs 75.6%).

RQ2: S2 level 13 was selected by a matching procedure (weighted log-ratio distance on area and cell count; score 0.319 vs 0.951 for the next candidate), not guessed. It edges H3-8 on skill (0.274 vs 0.256) — but its cells are still **47% larger**, which plausibly accounts for the entire gap. *Resolution is the decision that matters.*

7. Models and the test set

LightGBM — objective tested not assumed (Poisson 1.5695 beat L1 1.5739); the old 500-round cap was genuinely binding, so it was raised and early stopping allowed to decide (final models stopped at 569–1,101 rounds); 10-trial Optuna search improved validation 1.5695 → 1.5544.

TFT (239,688 params) — gated residual networks, variable selection, LSTM encoder over 96 intervals, interpretable attention. **ST-GNN** (35,432 params) — 2 graph-attention layers over first-ring cell adjacency + GRU over 24 intervals. Both hand-written, trained on a free Kaggle T4 in ~23 GPU-minutes. Scaling statistics fitted on TRAIN only and baked into the exported bundle, so leaking through normalisation is structurally impossible.

TEST — opened exactly once (Nov–Dec 2024, 1,877,208 rows)

Model @ H3-8	Pickup MAE	Dropoff MAE	WAPE	Hotspot F1
LightGBM (tuned)	1.1408	1.1275	0.3573	0.7248
ST-GNN	1.2024	1.1978	0.3686	0.7260
TFT	1.2919	1.4969	0.3523	—
Seasonal Naive (yesterday)	1.8430	1.8253	0.5771	0.6265
Seasonal Naive (last week)	1.9575	1.9409	0.6130	0.6380

All three real models beat both baselines by **35–42%**. Degradation with horizon is mild: a one-hour forecast is only **6% worse** than a 15-minute one.

Answering RQ3 carefully

At H3-8, TFT won WAPE (0.3523 vs 0.3573) and ST-GNN won Hotspot F1 (0.7260 vs 0.7248) — both landing exactly where their architectures predict. **Both reversed at S2-13**, where LightGBM took MAE, WAPE and Hotspot F1 outright.

WHY THAT MATTERS

Two sub-1% margins that flip under a change of tiling scheme are **noise, not evidence**. Stopping at H3-8 would have produced a more interesting story that happens to be wrong. Both are still reported — suppressing them would be selective.

THE HONEST CAVEAT

The deep models were stopped at **8 epochs and all four were still improving** on their final epoch. LightGBM ran to convergence. So the claim is "*a tuned LightGBM beats an 8-epoch TFT and ST-GNN*", not "gradient boosting beats deep learning". LightGBM is clearly the better *value* — 48 minutes of CPU and it won — but the deep models weren't given their best shot.

Where the model fails

Breakdown	Finding
By hour	Worst at 16:00 (MAE 1.809), best at 03:00 (0.281). Error tracks demand level.
By demand tercile	MAE 0.40 low / 0.84 mid / 2.75 high — but <i>relative</i> error is 62% mid vs 26% high. Which metric you quote decides which cells look broken.
Rain	MAE 0.85 wet vs 1.18 dry — not skill . Mean demand is 1.57 wet vs 3.40 dry; rain suppresses cycling, and smaller counts carry smaller errors.
Top-100 errors	73% are under-forecasts ; 38 occur at 16:00; all fall in just 22 of 321 cells . The one characteristic failure: under-predicting afternoon-rush spikes in high-volume cells.

On validation, same-week Seasonal Naive beat same-day and was adopted as the baseline. On TEST the order *reverses* — Nov–Dec contains Thanksgiving and Christmas, which break weekly patterns more than daily ones. Both are reported.

8. Operations — and a negative result

No dock-occupancy data exists for 2023–24, so capacity is estimated from the 95th percentile of daily cumulative net-flow range: **median 15 docks** across 1,496,558 station-days. 1.73% of station-days are infeasible without assuming staff intervened, and are flagged rather than forced to fit.

Forecast driving the rebalancer	Service level	Unserved	Moves
No rebalancing	0.9958	8,540	0
Perfect foresight	0.9940	12,294	2,615
LightGBM	0.9942	11,904	2,107
Seasonal Naive	0.9939	12,391	2,699

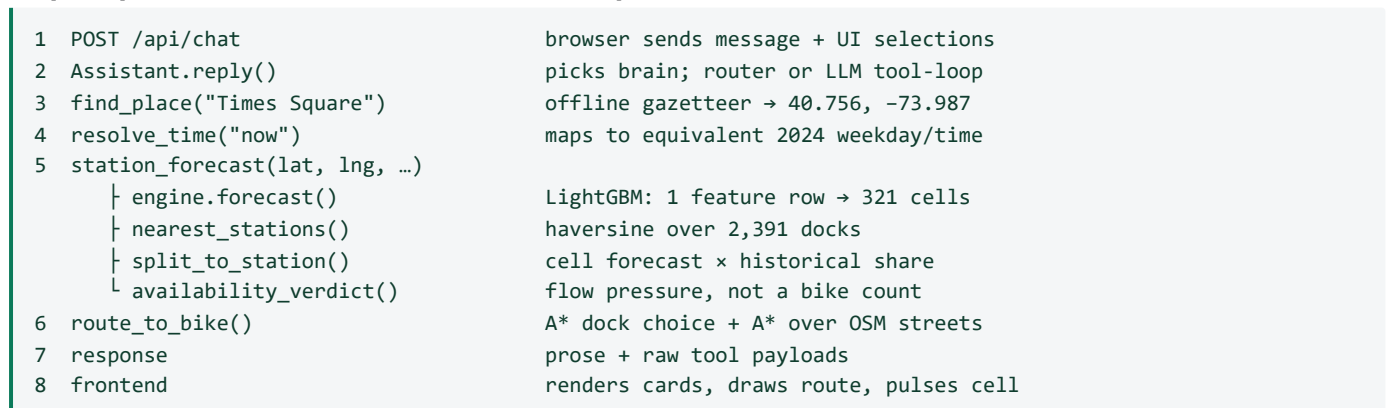
- (a) Among forecasts, LightGBM beat even perfect foresight** — 11,904 unserved vs 12,294. Not a paradox: LightGBM under-forecasts (73% of its worst errors), so it triggers **2,107 moves instead of 2,615**. Where moving is harmful, restraint wins.
- (b) Rebalancing itself doesn't pay**, and the cause is granularity. 321 cells aggregate 2,459 stations — ~115 docks per cell against ~3 trips per interval. **A cell that large cannot run dry**, so doing nothing already serves 99.58% and every move strips bikes from cells that need them later. Proving rebalancing value needs *station-level* inventory data that doesn't exist. Reporting a positive result here would have required tuning thresholds until the sign flipped.

9. System architecture

Layered view



Request path — "will I find a bike near Times Square?"



Because the API endpoints and the assistant call the **same tool functions**, the map and the chat can never show different numbers.

All five models serve from one engine — LightGBM 0.3 s warm, ST-GNN 0.17 s, TFT 0.40 s. The deep models need a separate serving path because they consume a *tensor window* of history (24 intervals for the graph model, 96 for the transformer), not a single feature row.

Three approximations, each labelled in the UI

- **Cell** → **station**. Models forecast ~0.74 km² cells holding ~8 stations. Station numbers are the cell forecast × that station's historical share of its cell's trips. Shares sum to exactly 1.000 across all 321 cells. Flagged `approximate: true` everywhere.
- **Demand, not availability**. We forecast *trips*, not dock occupancy, so the app reports **flow pressure** ("bikes leaving fast" / "balanced" / "docks filling up") rather than inventing a bike count.
- **Nothing is live**. Models cover 2023–24, so "now" maps to the equivalent 2024 weekday and time, labelled *typical conditions*.

Places without a maps API

Geocoding uses **our own data**: 2,459 station names (mostly NYC intersections) plus 19,011 landmarks already geocoded by the events pipeline. Index builds in 0.19 s. Matches below a confidence floor are flagged — "Wall Street" genuinely fails, because Citi Bike names that area differently, and admitting that beats teleporting the user to Brooklyn.

The honesty rule in the assistant

WHY THE CHATBOT WON'T BLAME A STREET FAIR

The natural thing for a chatbot to say is "*demand is high because there's a festival nearby*". That would be a **fabrication** — the ablation tested events twice, the second time with 4× the coverage, and measured −0.01%. The deployed model doesn't use events at all.

So tools return `drivers` (features the model genuinely uses) separately from `context` (events, weather — each carrying `influences_forecast: false`). A real explanation reads: "*the last hour averaged 91.0 pickups per 15 min, 27% above this cell's usual Wednesday 17:00 level; the same slot last week saw 151.*"

Three interchangeable brains — **Mistral**, **Claude**, or a **deterministic router** that needs no key so a fresh clone works with zero setup. All return the same payload shape; the LLM paths degrade to the router on any failure.

Walking directions — two graphs, two jobs

Which dock? A station graph where candidates are scored on `walking_metres + availability_penalty` — a draining dock costs up to 550 virtual metres, so a dock 200 m further with bikes arriving can win. *That is the point of putting a forecast behind a router.*

How do you walk there? The walkable street network pulled from **OpenStreetMap** via Overpass, fed to the *same* A*. Identical algorithm, real edges. The heuristic is great-circle distance — **admissible**, so A* still finds the optimal path.

Times Sq → W 45 St & 8 Ave	Straight line (before)	Street network (now)
Distance	387 m estimated	303 m measured
Waypoints	2	17, following real streets
Timing	instant	~4 s first, 0.11 s cached

The old estimate was also *wrong* — straight-line × 1.3 overstated by 28%. If Overpass is unreachable it falls back with `follows_streets: false`, and that flag reaches the UI.

Interface

Map-first with a sliding bottom sheet. Four keyless vector basemaps (OpenFreeMap Liberty by default — full buildings and POIs); `MAPTILER_KEY` unlocks more. Vector rather than raster specifically so the demand overlay sits *beneath the label layers* and street names stay readable. The heatmap is **off by default** — searching pulses just the one cell — and 2,391 real Citi Bike docks are drawn with their names.

10. Bugs worth remembering

Bug	What happened and how it was caught
Silent-drop in de-duplication	<code>WHERE key NOT IN (...)</code> evaluates to NULL — not TRUE — for a NULL key, so those rows were dropped with no rule attributed . Caught by the waterfall failing to reconcile: $7 + 3 \neq 12$.
Partitioned write discarded 97%	Polars names every partition file <code>00000000.parquet</code> and rewrites the directory, so each batch replaced the last — only 35 of 1,483 cells survived. Now writes via DuckDB and asserts on-disk counts match.
Validation on the wrong slice	Scoring the first N anchors meant scoring empty overnight intervals — reported MAE 0.665 where truth was 1.271.
Units mismatch in comparison	Showed 9 predicted vs 105 actual: station-level cumulative against cell-level single-interval. Two mismatches at once, understating every model $\sim 4\times$.
Station shares all zero	Registry stores counts as <code>Decimal(scale=0)</code> , so <code>302999/2034403</code> truncated to 0. Every station forecast was zero.
A stray word to Queens	"why is that the case?" left the word "case", which matched " Case St & 94 St ". Follow-ups now need a strong match to move the location.

11. Limitations

Data	Models & app
- No dock occupancy exists — capacity and inventory are estimates, flagged everywhere. - Traffic covers arterials only; 46 of 128 links within 500 m of a station. 45 days of outages. - Events are permits only — no MSG or Barclays; 25% of locations couldn't be geocoded. 255 stations that really operated are missing from the official roster.	- Deep models undertrained (8 epochs, still improving). - Training subsamples rows; validation and test never do. - No per-station model — station answers are disaggregated. - MAPE deliberately absent: most cells are zero. - Nothing live; runs locally, not deployed.

Highest-value next step: wire in Citi Bike's live **GBFS feed** — free, no key, gives real bikes-available per station. That converts availability from an inference into a measurement. Then train the deep models to convergence to settle RQ3 properly.

12. Running it

```
python -m uvicorn app.server:app --port 8000      # → http://127.0.0.1:8000
python -m pytest tests/ -q                        # 166 passing

# optional – set in the environment, never in the repo
export MISTRAL_API_KEY="..."    # or ANTHROPIC_API_KEY; else the router answers
export MAPTILER_KEY="..."       # optional extra basemaps
```

IN CLOSING

The most valuable output isn't the 1.14 MAE. It's the decisions where the honest answer was less impressive than the available one: that rebalancing didn't help, that two apparent deep-learning wins were noise, that traffic and weather and events earned nothing across two separate tests, and that we can't tell you how many bikes are at a dock because nobody published that data.

Every caveat here exists so the numbers that *are* claimed can be relied on.

13. Summary for a CV

One-line version

Built an end-to-end spatiotemporal ML system forecasting NYC bike-share demand from 79M trips, benchmarking gradient boosting against a Transformer and a Graph Neural Network across two spatial indexing schemes, and deployed it as an agentic map application with A* routing.

Bullet version

- Built a **79M-row spatiotemporal forecasting pipeline** (Polars + DuckDB + Parquet) processing 15 GB of raw trip data into a dense 104M-row panel; achieved **1.14 MAE on a held-out test set, 38% better than the seasonal baseline**.
- Trained and benchmarked **four model families across two spatial systems** — LightGBM (Optuna-tuned, Poisson objective), a hand-written Temporal Fusion Transformer, a Spatio-Temporal Graph Neural Network with graph attention, and Seasonal Naive — on a six-config matrix over Uber H3 and Google S2 grids.
- Identified that **standard accuracy metrics are invalid for comparing spatial resolutions** (MAE, WAPE and Hotspot-F1 gave three different winners) and introduced a baseline-relative skill score that removes the scale effect.
- Designed a **feature ablation over 168 features in 8 families**, proving that weather, events and traffic contribute ~0% — re-running the study at 4× traffic coverage to confirm the negative result rather than accept it at face value.
- Prevented data leakage with **35 automated tests that re-derive every lag feature by an independent join**; 166 tests total, with phase gates verified by scripts reading the produced artifacts.
- Deployed a **FastAPI + MapLibre application with an LLM tool-calling agent** (Mistral / Claude, plus a dependency-free deterministic fallback), including **A* pathfinding with an admissible heuristic over an OpenStreetMap walking graph**.
- Trained deep models on a **free Kaggle T4 GPU** by exporting a 557 MB tensor bundle, shipping the model definitions alongside the data so the remote run could not diverge from the repository.

What this demonstrates

Area	Evidence in this project
Data engineering at scale	15 GB → 2.5 GB in 2 min; 104M-row panel built in region batches with 4 KB spill instead of 12 GB; out-of-core SQL throughout on a 16 GB machine
ML fundamentals	Chronological splits, a test set opened exactly once, leakage tests by independent re-derivation, objective selection by experiment, metric validity analysis
Deep learning	TFT and ST-GNN implemented from scratch in PyTorch — gated residual networks, variable selection, graph attention via edge-list segment softmax
Software engineering	Config-driven, protocol-based spatial abstraction, 166 tests, one commit per phase, scripts that verify their own outputs
Product sense	Turning a cell-level forecast into a station-level answer with the approximation labelled; reporting flow pressure rather than inventing a bike count
Scientific integrity	Published four negative or partial results — rebalancing did not help, two deep-learning wins were noise, external features earned nothing — rather than tuning until they looked good

Talking points for an interview

- "Why is MAE the wrong metric here?"** — the cross-resolution analysis in §6 is the strongest single result and shows metric literacy beyond running `sklearn.metrics`.
- "Tell me about a bug you found."** — the NULL-key silent drop (§10) was caught by an arithmetic identity deliberately built into the cleaning step, not by luck.
- "When did the data surprise you?"** — LightGBM out-performing a *perfect* forecast at rebalancing (§8), because under-forecasting made it act less often.
- "How do you keep an LLM honest?"** — tools return caveats attached to the data, so the model cannot describe events as a cause when the ablation measured them at −0.01%.

A 43-page version with full derivations, all eight feature families, complete metric tables and a glossary is available as [docs/GeoPulse_Report.pdf](#).